Correção · Algoritmos · Árvores · 10 questões (diagnóstico)

Data: 11/11/2026 (bloco de 10/11) · **Semana 3** · **Placar: 5/10 (50%)** · 5 erros · 0 "não sabia"

Respostas lidas das anotações do PDF: 1-D · 2-A · 3-E · 4-B · 5-D · 6-A · 7-B · 8-C · 9-E · 10-E

> Questões autorais no estilo FGV, elaboradas para treino. Não são questões oficiais.

> **Como li as marcas:** usei os círculos nas letras (a tabela "Minhas respostas" ficou vazia). **Nenhum "?" e nenhum pontinho** em alternativa: as 10 foram marcadas com certeza. Os dois pontinhos soltos (ao lado do N, na Q1, e no rodapé da última página) não estão perto de nenhuma alternativa e foram tratados como toque acidental. Na Q7, o círculo na (B) está meio torto, mas é claramente na (B).

Questão	Assunto	Sua	Gabarito	Resultado
1	Grau, altura e folhas	D	C	❌
2	Inserção em BST + pré-ordem	A	A	✅
3	Percurso em ordem = crescente	E	E	✅
4	BST degenerada × AVL	B	B	✅
5	AVL: fator de balanceamento	D	D	✅
6	Máximo de nós e folhas (altura 4)	A	A	✅
7	Remoção com sucessor em ordem	B	D	❌
8	Caminho da busca em BST	C	E	❌
9	Função recursiva em Java (código)	E	B	❌
10	Arestas, menor valor da BST, n-ária (I/II/III)	E	C	❌

Por tipo:

Tipo	Questões	Placar
Percursos (pré, em, pós-ordem)	2, 3	**2/2**
Balanceamento, AVL e complexidade	4, 5	**2/2**
Contagem e propriedades (folhas, arestas, máximo de nós)	1, 6, 10	1/3
Operações na BST (buscar, remover)	7, 8	**0/2**
Leitura de código recursivo	9	0/1

Acertos firmes: 5 de 5. **Erros convictos:** 5 de 5 (nenhum com "?" ou pontinho).

Comparação: na mesma disciplina, Big-O (06/11) **7/12 (58%)** → listas, pilhas e filas (11/11) **8/10 (80%)** → árvores **5/10 (50%)**. O que ficou da faculdade é o **"desenho" da árvore**: você montou a BST da Q2 certinho, acertou os percursos e as fórmulas 2^h e 2^(h+1) − 1 da Q6. O que falta é **operar** sobre ela: buscar (Q8), remover (Q7) e ler o que uma função recursiva faz (Q9). E os 5 erros vieram **sem nenhuma marca de dúvida**, o mesmo padrão do simulado (ponto fraco 0a): a sensação de "sei isso" chegou antes de desenhar.

Teoria mínima do assunto

> Esta seção é a teoria do assunto. Como o diagnóstico saiu 5/10, árvores **não sai da fila** só com ela: falta um bloco curto de treino (ver "O que estudar agora").

1. Vocabulário (o que a FGV pede para contar)

Termo	Definição	Na árvore da Q1
Grau de um nó	número de filhos dele	J = 3, A = 2, M = 1, F = 0
Grau da árvore	maior grau entre todos os nós	3
Folha	nó **sem filhos** (grau 0), **em qualquer nível**	B, C, **F**, P, Q = 5
Nó interno	nó com pelo menos 1 filho	J, A, M, N = 4
Nível	distância (em arestas) até a raiz; raiz = 0	P e Q no nível 3
Altura	arestas do caminho mais longo raiz → folha	J → M → N → P = 3
Arestas	sempre **n − 1** (todo nó, menos a raiz, tem 1 pai)	9 nós → 8 arestas

📌 Para contar folhas, **percorra nó por nó** e pergunte "tem filho?". Não procure folhas só no último nível.

2. Árvore binária: limites (altura h, raiz no nível 0)

O quê	Fórmula	h = 4
Máximo de nós no nível k	2^k	nível 4 → 16
Máximo de folhas	2^h	16
Máximo de nós	2^(h+1) − 1	31
Mínimo de nós	h + 1 (árvore "em fila")	5

> ⚠️ Se a banca disser que a raiz está no **nível 1** (ou que a altura conta **nós**), tudo desloca uma casa: máximo de nós vira 2^h − 1. **Leia a convenção no enunciado.**

3. Árvore binária de busca (BST): as três operações

Operação	Como faz	Custo
Buscar x	compara com o nó atual: menor → **só** esquerda, maior → **só** direita. Nunca visita o irmão	O(altura)
Inserir x	busca até cair num `null` e pendura ali (o novo nó é sempre folha)	O(altura)
Remover x	0 filhos: apaga · 1 filho: o filho sobe · **2 filhos: troca pelo sucessor** (menor da direita) **ou pelo antecessor** (maior da esquerda)	O(altura)
Menor / maior	desce sempre à esquerda / sempre à direita até não dar mais	O(altura)

- **Sucessor em ordem** = próximo valor na ordem crescente = **um passo à direita e depois tudo à esquerda**.
- **Antecessor em ordem** = **um passo à esquerda e depois tudo à direita**.

- O menor nó **não tem filho esquerdo**, mas **pode ter filho direito** → não é necessariamente folha.

4. Percursos

Percurso	Ordem	Uso típico
Pré-ordem	**Raiz**, esquerda, direita	copiar/serializar a árvore (refazendo as inserções)
Em ordem	esquerda, **Raiz**, direita	**BST → ordem crescente**
Pós-ordem	esquerda, direita, **Raiz**	apagar a árvore, calcular tamanho de pastas
Em nível (largura)	nível a nível, com **fila**	menor caminho, imprimir por nível

5. Balanceamento

Árvore	Busca no pior caso	Por quê
BST sem balanceamento	**O(n)**	chaves em ordem (crescente ou decrescente) viram uma "lista"
AVL	**O(log n)**	em todo nó, |altura esq − altura dir| ≤ 1; rebalanceia com **rotações** na inserção e na remoção
Rubro-negra	O(log n)	balanceamento mais frouxo, menos rotações (TreeMap do Java)
Heap	busca O(n)	não é BST: só garante pai ≥ filhos (ou ≤); serve para fila de prioridade

6. Recursão em árvore: o molde que a FGV usa

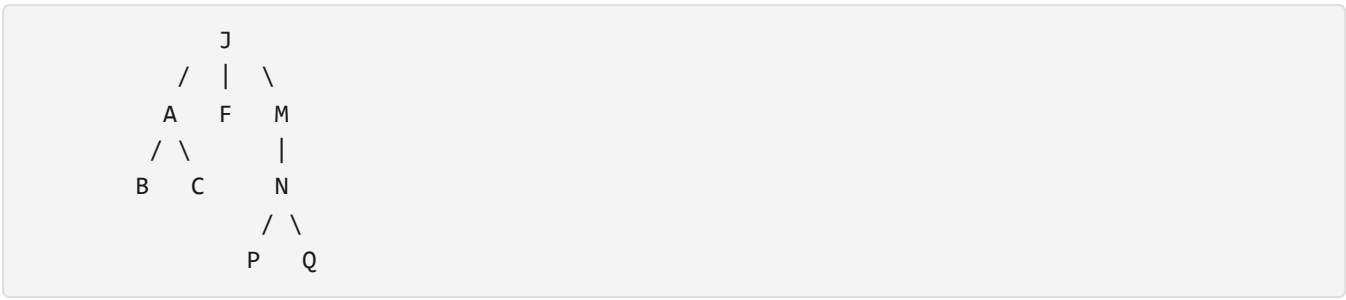
```
int f(No n) {
    if (n == null) return A;                          // caso vazio
    if (n.esq == null && n.dir == null) return B;     // caso folha (opcional)
    return C(f(n.esq), f(n.dir));                     // combina os filhos
}
```

A (null)	B (folha)	C (combina)	Calcula
0	—	1 + esq + dir	**número de nós**
0	**1**	esq + dir	**número de folhas**
−1	—	1 + max(esq, dir)	**altura** (em arestas)
0	—	n.valor + esq + dir	soma dos valores

📌 **Leia o que a linha da folha devolve.** Se só a folha devolve 1 e o resto só soma, a função conta folhas.

Questão 1 · ❌ Errou

O sistema SIGEP-PB organiza a estrutura administrativa do Tribunal na árvore a seguir, em que J é a raiz:



O grau da árvore, a sua altura e o número de folhas são, respectivamente:

(A) 2, 3 e 5;

(B) 3, 4 e 5;

✅ **(C) 3, 3 e 5;**

❌ (D) 3, 3 e 4; ← sua resposta

(E) 9, 3 e 5.

Onde você se confundiu: grau (3) e altura (3) estavam certos, e você escreveu "3 3 4" na margem. O erro foi na contagem das folhas: você circulou **B, C, P e Q** e deixou de fora o **F**. O F está no nível 1, logo abaixo da raiz, e **não tem filho nenhum**: é folha. A procura parou nos "pés" da árvore (níveis 2 e 3), mas folha não depende de nível, depende de não ter filho.

Por que as outras erram: (A) dá grau 2 (é o grau do A, não o da árvore, que é o maior, o do J) · (B) dá altura 4 (contou **nós** do caminho J-M-N-P, não arestas) · (E) dá grau 9 (é o número de nós).

📌 **Folha = nó sem filhos, em qualquer nível.** Para contar, passe por **todos** os nós (J, A, F, M, B, C, N, P, Q) e marque os que não têm "perninha" para baixo. Conferência rápida: folhas + nós internos = total (5 + 4 = 9).

Questão 2 · ✅ Acertou

O analista Rafael inseriu, nesta ordem, as chaves 50, 30, 70, 20, 40, 60, 80 e 35 em uma árvore binária de busca inicialmente vazia, sem nenhum balanceamento, no sistema e-PRAZOS.

O percurso em pré-ordem da árvore resultante é:

✅ **(A) 50, 30, 20, 40, 35, 70, 60, 80;** ← sua resposta

(B) 20, 30, 35, 40, 50, 60, 70, 80;

(C) 20, 35, 40, 30, 60, 80, 70, 50;

(D) 50, 30, 70, 20, 40, 60, 80, 35;

(E) 50, 70, 80, 60, 30, 40, 35, 20.

Por quê: você desenhou a árvore certa (35 à esquerda do 40) e eliminou bem pelo primeiro elemento: a pré-ordem começa **sempre pela raiz** (50), o que derruba (B) e (C). A (B) é a em ordem (crescente), a (C) é a pós-ordem, a (D) é a ordem de inserção (aqui coincide com o percurso em nível) e a (E) visita a direita antes da esquerda.

📌 Pré-ordem = **Raiz, esquerda, direita**: o 1º é sempre a raiz. Pós-ordem: a raiz é a **última**. Em ordem de uma BST: crescente.

Questão 3 · ✅ Acertou

O sistema e-CONSULTA mantém os números dos processos de uma vara em uma árvore binária de busca, em que valores menores ficam à esquerda. A analista Tânia precisa gerar um relatório com todos os números em ordem crescente, sem executar nenhum algoritmo de ordenação depois de percorrer a árvore.

Para isso, a analista deve usar o percurso:

(A) em pré-ordem, pois ele visita primeiro a raiz, que contém o menor valor da árvore;

(B) em pós-ordem, pois ele visita primeiro as folhas, que contêm os menores valores;

(C) em nível (largura), pois ele visita os nós da esquerda para a direita, nível a nível;

(D) em pré-ordem, visitando a subárvore direita antes da subárvore esquerda;

✅ **(E) em ordem (in-order), pois ele visita a subárvore esquerda, depois a raiz e depois a subárvore direita.** ← sua resposta

Por quê: esquerda (menores) → raiz → direita (maiores) dá a ordem crescente. A (A) erra porque a raiz não é o menor valor, a (B) porque as folhas não guardam necessariamente os menores, a (C) porque o nível mistura valores (na Q2 daria 50, 30, 70...) e a (D) daria uma ordem sem padrão.

📌 **BST + em ordem = crescente.** Em ordem com a direita antes da esquerda = decrescente.

Questão 4 · ✅ Acertou

No sistema e-PROTOCOLO, os números de protocolo são gerados em sequência crescente e inseridos, à medida que chegam, em uma árvore binária de busca simples, sem balanceamento. Após alguns meses, com centenas de milhares de protocolos, o analista Eduardo notou que as buscas ficaram muito lentas.

A explicação correta e uma solução adequada são:

(A) a árvore binária de busca tem busca O(log n) em qualquer caso, e a lentidão deve ter outra causa, como o banco de dados;

✅ **(B) as chaves inseridas em ordem crescente degeneram a árvore em uma lista, com busca O(n), e uma árvore AVL manteria a busca em O(log n);** ← sua resposta

(C) as chaves inseridas em ordem crescente geram uma árvore completa, com busca O(n log n), e uma heap resolveria o problema;

(D) a árvore degenerou, com busca $O(n^2)$, e uma lista ordenada resolveria o problema, com busca O(1);

(E) as chaves inseridas em ordem crescente geram uma árvore cheia, e a troca por uma árvore AVL não alteraria a complexidade da busca.

Por quê: você sublinhou as três pistas ("sequência crescente", "BST simples", "sem balanceamento") e riscou a palavra errada de cada distratora ("qualquer", "completa", "$O(n^2)$", "cheia"). Técnica exata. Cada nova chave é maior que todas e vai sempre para a direita: a árvore vira uma fila de nós.

📌 BST sem balanceamento + chaves **ordenadas** = degenera em lista → busca **O(n)**. AVL (ou rubro-negra) garante **O(log n)**. "Em qualquer caso" é a palavra que derruba.

Questão 5 · ✅ Acertou

A equipe da analista Patrícia avalia usar uma árvore AVL como índice em memória do módulo de agendamento do sistema e-AUDIÊNCIA, em que há muitas inserções e buscas.

Sobre a árvore AVL, é correto afirmar que:

(A) o fator de balanceamento de cada nó pode assumir qualquer valor, desde que o fator de balanceamento da raiz seja 0;

(B) ela dispensa a regra da árvore binária de busca, pois a posição de cada nó é definida apenas pelo balanceamento;

(C) ela garante busca em O(1), pois todas as suas folhas ficam sempre no mesmo nível;

✅ **(D) em todo nó, a diferença entre as alturas das subárvores esquerda e direita é −1, 0 ou 1, o que mantém busca, inserção e remoção em O(log n);** ← sua resposta

(E) ela só se rebalanceia nas remoções, e, nas inserções, comporta-se como uma árvore binária de busca comum.

Por quê: AVL é uma BST (não dispensa a regra, contra a B) com fator de balanceamento −1, 0 ou 1 em **todo** nó (não só na raiz, contra a A). Isso dá O(log n), não O(1), e as folhas podem ficar em níveis diferentes (contra a C). Rebalanceia com rotações na inserção **e** na remoção (contra a E).

📌 **AVL = BST + |h(esq) − h(dir)| ≤ 1 em todo nó** → buscar, inserir e remover em O(log n), com rotações nas duas operações de escrita.

Questão 6 · ✅ Acertou

A equipe do analista Leonardo vai armazenar os dados de um cadastro do sistema e-PERITOS em uma árvore binária de altura 4 (raiz no nível 0).

O número máximo de nós e o número máximo de folhas que essa árvore pode ter são, respectivamente:

✅ **(A) 31 e 16;** ← sua resposta

(B) 16 e 8;

(C) 15 e 8;

(D) 32 e 16;

(E) 31 e 15.

Por quê: você escreveu as duas fórmulas (2^h = 16 folhas; 2^(h+1) − 1 = 31 nós) e ainda desenhou as árvores crescendo nível a nível para conferir. A (C) usa altura 3, a (D) esquece o "− 1" e a (E) troca as folhas por 2^h − 1.

📌 Binária de altura h (raiz no nível 0): **máx. folhas = 2^h** · **máx. nós = 2^(h+1) − 1**. Conferência: o último nível tem tantos nós quanto todos os de cima somados, mais 1 (16 = 15 + 1).

Questão 7 · ❌ Errou

A árvore binária de busca do sistema e-CUSTAS foi montada com a inserção, nesta ordem, das chaves 45, 25, 65, 15, 35, 55, 75, 50 e 60. O analista Fernando precisa remover a chave 45, que está na raiz e tem dois filhos, usando o **sucessor em ordem** para ocupar o lugar do nó removido.

Após a remoção, a nova raiz da árvore será:

(A) 35, que é o maior valor da subárvore esquerda;

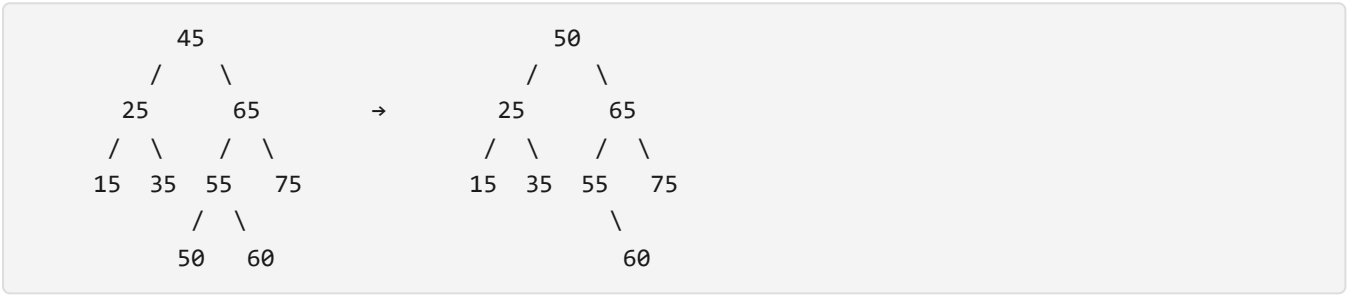
❌ (B) 65, que é o filho direito da raiz removida; ← sua resposta

(C) 75, que é o maior valor da árvore;

✅ **(D) 50, que é o menor valor da subárvore direita;**

(E) 25, que é o filho esquerdo da raiz removida.

Onde você se confundiu: você sublinhou certo "45", "dois filhos" e "sucessor em ordem", mas **não desenhou a árvore** (na Q2 desenhou e acertou) e tomou "sucessor" como "o filho que vem logo depois, à direita". Sucessor **em ordem** é o **próximo número na ordem crescente**: depois do 45, quem vem? Em ordem, a árvore dá 15, 25, 35, **45, 50**, 55, 60, 65, 75. O próximo é o **50**.



Para achar sem escrever a sequência: **um passo à direita (65) e depois tudo à esquerda (55 → 50)**. O 50 não tem filho esquerdo, então sai do lugar sem complicação.

Por que a (B) não funciona: se o 65 subisse para a raiz, o 55 e o 60 (menores que 65) teriam de ficar à esquerda dele, junto com o 25, e a árvore teria de ser remontada. O sucessor é escolhido justamente porque cabe na raiz sem quebrar a regra da BST: é maior que tudo à esquerda e menor que tudo à direita. A (A) é o **antecessor** (seria a resposta se o enunciado pedisse antecessor), a (C) e a (E) não guardam a ordem.

📌 **Remover nó com 2 filhos:** substituir pelo **sucessor** (menor da direita: um passo à direita, depois tudo à esquerda) **ou pelo antecessor** (maior da esquerda: um passo à esquerda, depois tudo à direita). Nunca é simplesmente o filho direito, salvo se ele não tiver filho esquerdo.

Questão 8 · ❌ Errou

O sistema e-URGENTE guarda os códigos dos pedidos de liminar em uma árvore binária de busca, sem balanceamento. A árvore foi montada com a inserção, nesta ordem, das chaves 60, 40, 80, 20, 50, 70, 90, 45 e 55. A analista Débora fez uma busca pela chave 55.

A sequência de nós visitados pela busca, da raiz até encontrar a chave, é:

(A) 20, 40, 45, 50, 55;

(B) 60, 40, 80, 20, 50, 70, 90, 45, 55;

❌ (C) 60, 40, 20, 50, 45, 55; ← sua resposta

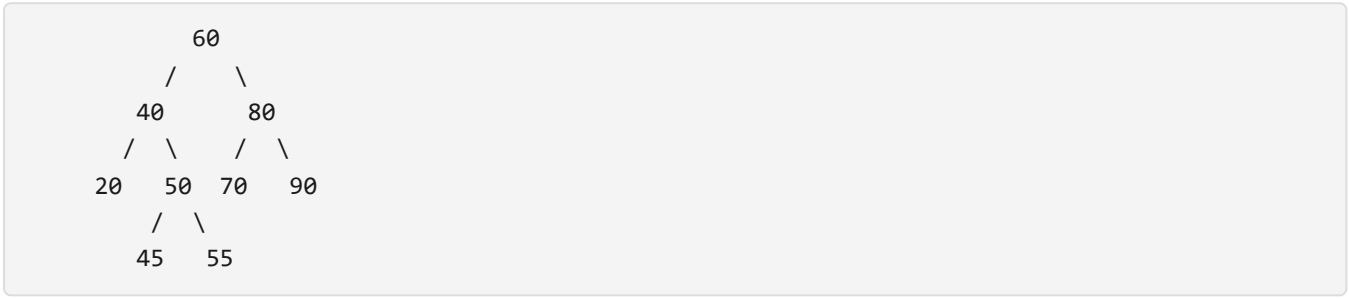
(D) 60, 40, 50, 45, 55;

✅ **(E) 60, 40, 50, 55.**

Onde você se confundiu: as suas marcas mostram o raciocínio: você riscou o "20" da (A) (não começa pela raiz), o "80" da (B) (foi para a direita) e o **"50" da (D) e da (E)**. Depois do 40 você esperava o **20**. A sua (C) é **exatamente uma busca em profundidade em pré-ordem** (raiz, esquerda, direita, parando ao achar o 55),

e a (B) é a busca em largura. As duas são buscas **corretas para uma árvore qualquer**, e você executou a sua sem erro. O que faltou foi usar a **ordenação** da árvore binária de busca, que é o motivo de ela existir.

> **Nota sobre o enunciado (reclamação do Tiago em 11/11, procedente em parte):** o enunciado não nomeia o algoritmo. A convenção da área (e da FGV) é que "busca em árvore binária de busca" significa o algoritmo próprio dela, que compara e desce por um lado só. Por isso o gabarito continua (E), e um recurso dificilmente seria aceito. Mas a redação poderia fechar a porta: "utilizando o algoritmo de busca da árvore binária de busca". Como (B) e (C) são buscas genéricas válidas, a questão cobra justamente essa convenção. Na prova real, quando aparecer "árvore binária de busca" e "busca", **use a ordenação**: DFS e BFS só entram quando o enunciado falar em profundidade, largura ou numa árvore sem ordem.



Busca por 55: 55 < 60 → esquerda (**40**) · 55 > 40 → direita (**50**) · 55 > 50 → direita (**55**) · achou. O 20 e o 45 **nunca** são visitados: estão do lado que a comparação descartou. A (D) erra por esse mesmo motivo (passa no 45, que é menor que 50), a (B) é a ordem de inserção e a (A) começa por uma folha.

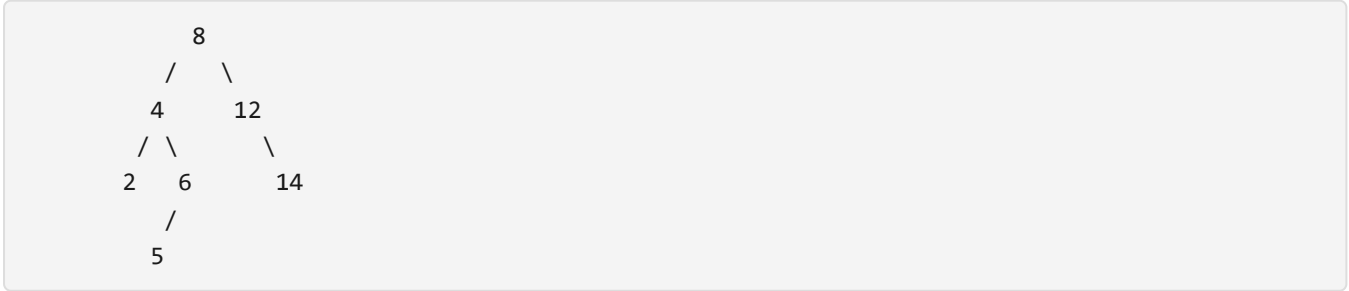
📌 **Busca na BST = uma comparação por nível e UM lado só.** Menor vai à esquerda, maior vai à direita, e o outro lado é descartado inteiro. O caminho tem no máximo **altura + 1** nós e nunca tem dois irmãos. Teste rápido nas alternativas: se aparecem dois filhos do mesmo nó (20 e 50, ambos filhos do 40), está errada.

Questão 9 · ❌ Errou

O analista Caio encontrou o método recursivo a seguir no sistema SISDOC, escrito em Java:

```
int f(No n) {
    if (n == null) return 0;
    if (n.esq == null && n.dir == null) return 1;
    return f(n.esq) + f(n.dir);
}
```

O método foi chamado com a raiz da árvore binária a seguir:



O valor retornado e o que o método calcula são, respectivamente:

(A) 7, o número de nós da árvore;

✅ **(B) 3, o número de folhas da árvore;**

(C) 3, a altura da árvore;

(D) 4, o número de nós internos da árvore;

❌ (E) 2, o número de nós com dois filhos. ← sua resposta

Onde você se confundiu: você **riscou a linha `if (n == null) return 0;`**, como se ela não fizesse nada, e não rastreou nenhuma chamada na margem (na prova de listas, que rastreou, fez 5/5). Provavelmente leu `n.esq == null && n.dir == null` e associou "esq e dir" a "nó com dois filhos". Mas a condição diz o contrário: esquerda **nula** e direita **nula** = **nenhum** filho = **folha**. E é a linha que devolve 1. Todas as outras só **somam** o que vem dos filhos, sem acrescentar nada.

Rastreio:

Chamada	Caso	Devolve
f(2)	folha	1
f(5)	folha	1
f(6)	f(5) + f(null) = 1 + **0**	1
f(4)	f(2) + f(6) = 1 + 1	2
f(14)	folha	1
f(12)	f(null) + f(14) = **0** + 1	1
f(8)	f(4) + f(12) = 2 + 1	**3**

A linha que você riscou é a que trata o 6 e o 12 (nós com **um** filho só): sem ela, `f(null)` tentaria ler `n.esq` de um nulo e o Java lançaria **NullPointerException**. As folhas são 2, 5 e 14. Os distratores trazem os valores certos das outras medidas, para quem sabe o número mas não a função: 7 nós, altura 3 (8-4-6-5), 4 internos (8, 4, 6, 12) e 2 com dois filhos (8, 4).

📌 **Recursão em árvore: olhe o que a linha da folha devolve.** Só a folha devolve 1 + o resto só soma = **conta folhas**. Contar nós = `1 + f(esq) + f(dir)`. Altura = `1 + max(...)`. **Nunca risque o caso base:** sem `if (n == null)`, a recursão quebra no primeiro filho ausente.

Questão 10 · ❌ Errou

Em um treinamento sobre estruturas de dados para a equipe de desenvolvimento de um Tribunal de Justiça, foram apresentadas as afirmativas a seguir.

I. Uma árvore com n nós tem exatamente n − 1 arestas, pois cada nó, exceto a raiz, tem exatamente um pai.

II. Em uma árvore binária de busca, o menor valor está sempre armazenado em uma folha.

III. Em uma árvore n-ária, cada nó pode ter mais de dois filhos; a árvore binária é o caso particular em que cada nó tem, no máximo, dois filhos.

Está correto o que se afirma em

(A) I, apenas.

(B) II, apenas.

✅ **(C) I e III, apenas.**

(D) II e III, apenas.

❌ (E) I, II e III. ← sua resposta

Onde você se confundiu: você deu ✓ nos três itens (bom: julgou um por um). I e III estão certos. O II parece verdadeiro porque, na maioria dos desenhos, o menor fica lá embaixo, à esquerda. Mas o menor é o nó **mais à esquerda**, e isso só garante que ele **não tem filho esquerdo**. Ele **pode ter filho direito**:

```
      50
     /
   30        ← menor valor da árvore
     \
      40     ← filho direito do 30
```

O 30 é o menor e não é folha. Bastava um contraexemplo de 3 nós.

Pista: "sempre" em afirmativa de estrutura de dados pede contraexemplo. Tente montar a menor árvore que quebra a frase antes de dar ✓.

📌 **Menor da BST = descer sempre à esquerda até não haver filho esquerdo.** Esse nó pode ter filho direito, então não é obrigatoriamente folha (o mesmo vale para o maior, que pode ter filho esquerdo). E toda árvore com n nós tem **n − 1 arestas**.

Para o caderno de erros

Assunto	Por que errei	Regra certa
Contar folhas (Q1)	Procurei folhas só nos níveis de baixo; esqueci o F, no nível 1	Folha = sem filhos, em qualquer nível. Passar por todos os nós
Remoção com sucessor (Q7)	Tomei "sucessor" pelo filho direito; não desenhei a árvore	Sucessor = próximo na ordem crescente = um passo à direita, depois tudo à esquerda
Busca na BST (Q8)	Visitei os dois filhos do 40 (percorri em vez de buscar)	Busca desce por um lado só; o caminho nunca tem dois irmãos
Recursão em árvore (Q9)	Risquei o caso base e li "esq == null && dir == null" como "dois filhos"	Os dois nulos = folha. Só a folha devolve 1 e o resto soma = conta folhas
Menor da BST (Q10, item II)	Aceitei o "sempre" sem testar contraexemplo	Menor = mais à esquerda; pode ter filho direito, então nem sempre é folha

---

O que estudar agora

Quando	O que fazer
Agora, 15 min	Ler a **teoria mínima** acima e refazer **Q7, Q8 e Q9 sem olhar**, desenhando a árvore em cada uma (na Q9, preencher a tabela de rastreio)
Almoço · Português	🔴 **G. Semântica e reescrita** (1º da fila de Português)
19h · Bloco 1	🔴 **APF funções de dados:** responder o bloco de 06/11 (8 questões)
20h · Bloco 2	🔴 **Árvores · bloco de treino** (6 a 8 questões só de buscar, remover, contar folhas e recursão, pedir ao Claude). Árvores **só sai da fila** com esse bloco corrigido. Hash (diagnóstico pronto) fica para depois
22:40 · revisão	Regras 📌 de hoje (listas e árvores) + revisão espaçada de 04/11 + 15 min de Anki
Dom 15/11 · 13h	Simulado de domingo (rotina do plano)

Hábito para a prova: em toda questão de árvore com chaves inseridas, **desenhe a árvore antes de ler as alternativas**. Nas duas em que você desenhou (Q2 e Q6) acertou. Nas três em que não desenhou (Q7, Q8 e Q9), errou.